API Security Risk Analysis Report

Cyber Security Task 3 — Future Interns Program (2026)

Modern SaaS Skill · Read-Only API Testing · Risk Classification · Remediation Guidance

Prepared by: Lakshya Mishra

Role: Cyber Security Intern (AppSec / API Security)

Date: June 2026

Table of Contents

1. Introduction & Objective

Modern SaaS products, mobile apps, and dashboards are built on APIs rather than traditional web pages, which means an API's security posture is often the real security posture of the whole product. An insecure API can expose customer data, allow one user to reach another user's records, or be abused at scale, regardless of how secure the front-end application looks.

This report documents a read-only API security risk analysis performed against two publicly available demo APIs that are explicitly intended for testing and learning. The objective was to assess authentication and access-control posture, identify common API risk patterns, classify their severity, and translate the findings into business language and concrete remediation steps — the same deliverable a paid AppSec consulting engagement would produce for a SaaS client.

Objectives

- Test public/demo APIs using read-only requests only, and document authentication requirements, headers, and response data.
- Identify common API risk patterns: open endpoints, excessive data exposure, weak authentication, authorization gaps, missing rate limiting, and input validation issues.
- Classify each finding's severity as Low, Medium, or High.
- Explain business impact in plain language and provide clear remediation steps.

2. Scope & Ethics

This analysis was deliberately scoped to stay within safe, ethical, and legal bounds, consistent with a professional AppSec engagement.

In Scope

- JSONPlaceholder (jsonplaceholder.typicode.com) — a free, public, intentionally-open mock REST API designed for testing and prototyping.
- ReqRes (reqres.in) — a public REST API for testing and prototyping, referenced for its authentication model.
- Read-only GET requests, and review of officially published documentation describing write-endpoint (POST/PUT/PATCH/DELETE) behaviour.

Out of Scope / Not Performed

- No exploitation, authentication bypass attempts, or unauthorized access of any kind.
- No flooding, load testing, or denial-of-service testing.
- No testing of any private, production, or third-party system not explicitly published as a public test API.

All findings below are based on observing the documented and actual behaviour of these public test APIs, used exactly as their providers intend them to be used.

3. Methodology & Tools Used

- Reviewed each API's official documentation to understand available endpoints, supported methods, and authentication model.
- Issued read-only GET requests to representative endpoints and captured the real response data, status codes, and content types returned.
- Attempted each request with no credentials supplied, to test whether authentication is actually enforced rather than just documented.
- Cross-checked write-endpoint (POST/PUT/PATCH/DELETE) behaviour against the API providers' own published documentation and issue trackers, without performing destructive testing.
- Mapped each observed pattern to the relevant OWASP API Security Top 10 category and classified business impact and severity.

Tools Referenced

API testing	Postman, Insomnia (recommended tools for hands-on request building); direct HTTP requests for this analysis
Inspection	Response body, status code, and content-type review; authentication-requirement testing
Reference reading	OWASP API Security Top 10 project; API Security Checklist — consulted for category structure only, no content reproduced
Documentation	Word / PDF report generation

4. APIs Tested

API 1	JSONPlaceholder — jsonplaceholder.typicode.com — free mock REST API, 6 resource types (posts, comments, albums, photos, todos, users), no authentication by design
API 2	ReqRes — reqres.in — public REST API for testing; as of current documentation, requires an x-api-key header on every request

JSONPlaceholder was used for hands-on read-only testing because it explicitly permits and encourages this kind of use. ReqRes is referenced primarily for its authentication model, since its current policy of requiring an API key on every request — including simple reads — serves as a useful contrast to JSONPlaceholder's fully open design.

5. Findings

Six findings are documented below: five derived from direct, read-only testing and one (Finding 6) included as a positive-control reference point. Each finding lists the evidence observed, a plain-language explanation, business impact, severity, and remediation.

Finding 1: Unauthenticated Endpoint Exposing Full Personal Data

API / Endpoint	JSONPlaceholder — GET /users
Authentication required	None
OWASP API category	API2: Broken Authentication / API3: Excessive Data Exposure

Evidence

```
Request:  GET https://jsonplaceholder.typicode.com/users
Headers sent: none (no Authorization, no API key)
Response: 200 OK, application/json

[
  {
    "id": 1, "name": "Leanne Graham", "username": "Bret",
    "email": "Sincere@april.biz",
    "address": { "street": "Kulas Light", "city": "Gwenborough",
                 "zipcode": "92998-3874",
                 "geo": { "lat": "-37.3159", "lng": "81.1496" } },
    "phone": "1-770-736-8031 x56442",
    "company": { "name": "Romaguera-Crona", ... }
  },
  ... 9 more full user records returned in the same response ...
]
```

A request to the /users endpoint with no credentials of any kind returns full records for all ten users in the dataset, including legal name, email address, phone number, full postal address, geographic coordinates, and employer details.

No token, key, or login of any kind was required to retrieve this. The endpoint also offers no field-level filtering, so the full object — not just the fields a typical client would need — is returned every time.

Business Impact

If this exact design were used in a production SaaS product, anyone with the API's base URL could harvest a complete customer or employee directory with a single request and no account. That data could feed phishing or social-engineering campaigns, be sold or scraped at scale, or trigger regulatory exposure under data-protection laws (e.g. India's DPDP Act, GDPR) that require personal data to be access-controlled.

Remediation

- Require authentication (API key, OAuth2 token, or session) on every endpoint that returns personal data — including read-only GET requests.
- Apply field-level response filtering so each client only receives the fields it actually needs (avoid returning full internal objects by default).
- Classify which fields count as personal data and apply stricter access logging and rate limiting specifically to those endpoints.

Severity: HIGH

Finding 2: Enumerable IDs With No Object-Level Authorization Check

API / Endpoint	JSONPlaceholder — GET /posts/{id}, GET /posts?userId={id}
Authentication required	None
OWASP API category	API1: Broken Object Level Authorization (BOLA)

Evidence

```
Request:  GET https://jsonplaceholder.typicode.com/posts/1
Response: 200 OK
{ "userId": 1, "id": 1, "title": "sunt aut facere repellat provident",
  "body": "quia et suscipit ..." }

IDs 1-100 are sequential and every one of them returns data with the
same unauthenticated request. Filtering by ?userId=<any value> also
succeeds for every user ID without checking who is asking.
```

Every post resource is reachable simply by incrementing a numeric ID in the URL, and the userId query filter accepts any value without verifying that the caller is entitled to see that user's data.

On this particular sandbox the data is intentionally public, so nothing is actually being breached. The value of the finding is the pattern: this is exactly the request shape that causes real damage in a production system.

Business Impact

In a real multi-tenant SaaS application, this pattern is what allows one customer to view or modify another customer's records simply by changing an ID in the request — Broken Object Level Authorization, consistently the most reported API vulnerability category in industry research. The business impact ranges from a single data leak to a mass cross-tenant breach, depending on how predictable the IDs are.

Remediation

- Enforce an object-level authorization check on every request: confirm the authenticated caller is permitted to access the specific record ID requested, not just that they are logged in.
- Prefer unpredictable identifiers (UUIDs) over sequential integers for resources that will be referenced directly in URLs, to reduce easy enumeration.
- Add automated tests that specifically try to access another user's ID with a different user's token, and fail the build if that succeeds.

Severity: **MEDIUM**

Finding 3: Write Responses That Don't Reflect Real Persistence

API / Endpoint	JSONPlaceholder — POST / PUT / PATCH / DELETE on /posts, /users
Authentication required	None
OWASP API category	API9: Improper Inventory Management (undocumented mock behaviour)

Evidence

```
Request:  POST https://jsonplaceholder.typicode.com/posts
  Body: { "title": "foo", "body": "bar", "userId": 1 }
Response: 201 Created, with a new id assigned

Behaviour confirmed via the project's own public issue tracker:
a follow-up GET for the same resource still returns only the original
data — the create/update/delete never actually persists server-side.
```

Write requests return success-looking status codes and well-formed response bodies, but the underlying dataset is never actually changed. A tester relying only on the HTTP status code would conclude the write succeeded when it did not.

Business Impact

This is a documentation/inventory risk rather than a security breach on this sandbox, but the underlying lesson applies broadly: automated test suites or integrations that only check status codes (not actual resulting state) can carry false confidence into production, masking real failures.

Remediation

- Always validate state changes with an independent follow-up read, not just the response status code, when testing or integrating against any API.
- Clearly document in any API's reference material whether a given environment is a persistent data store or a non-persistent mock/sandbox.

Severity: **LOW**

Finding 4: Loose Type Validation on Write Endpoints

API / Endpoint	JSONPlaceholder — POST /posts
Authentication required	None
OWASP API category	API8: Security Misconfiguration (input validation)

Evidence

```
Publicly reported behaviour (project issue tracker) shows that a
numeric field submitted in a request body can be echoed back in the
response as a different data type than what the documentation
specifies, with no validation error raised.
```

The API accepts and echoes back data without strictly enforcing the field types described in its own documentation. Nothing rejects malformed or mistyped input outright.

Business Impact

Loose type handling on write endpoints can let malformed input pass through silently in a real system, which downstream code may not be prepared to handle correctly — leading to data corruption, crashes in dependent services, or logic errors that are hard to trace back to their source.

Remediation

- Validate every incoming field against a strict schema (JSON Schema or an OpenAPI-defined request body) before any business logic runs.
- Reject requests with a clear 4xx error and message when a field's type or format doesn't match the contract, rather than silently coercing it.

Severity: **LOW**

Finding 5: No Rate Limiting Observed on Public Read Endpoints

API / Endpoint	JSONPlaceholder — GET /users, /posts, /comments, etc.
Authentication required	None
OWASP API category	API4: Unrestricted Resource Consumption

Evidence

```
Multiple sequential GET requests were issued during this analysis.
Every request returned 200 OK with no rate-limit headers
(e.g. X-RateLimit-*) and no 429 Too Many Requests response at any point.
```

Nothing in the observed responses indicates that request volume is being tracked or throttled per caller. This is expected and acceptable for a free public test sandbox, but the absence of any limiting at all is worth calling out as a pattern.

Business Impact

Without rate limiting, a real production API is exposed to automated scraping, credential-stuffing attempts against login endpoints, and resource-consumption abuse that can degrade service for legitimate users or drive up infrastructure cost — particularly dangerous when combined with Finding 1's unauthenticated personal-data exposure.

Remediation

- Apply per-IP and/or per-API-key rate limiting on all endpoints, with stricter limits on endpoints that return bulk or sensitive data.
- Return standard rate-limit headers and a proper 429 response so legitimate clients can back off gracefully.

Severity: **MEDIUM**

Finding 6: API-Key Enforcement Done Correctly (Reference / Positive Control)

API / Endpoint	ReqRes — all endpoints (api.reqres.in)
Authentication required	x-api-key header on every request
OWASP API category	Reference example — not a vulnerability

Evidence

```
ReqRes's own current documentation states that every request requires
an x-api-key header, and that unauthenticated requests are rejected
with a clear error pointing to the sign-up page for a key — confirmed
directly against the live documentation during this analysis.
Automated access to the live endpoint itself was correctly blocked
by the site for unauthenticated tooling, which is consistent with
this stated policy.
```

Unlike JSONPlaceholder, ReqRes now gates every single endpoint — including simple read requests — behind a required API key, and fails closed (rejects the request) rather than failing open when no key is present.

Business Impact

Included as a contrast case: this is the behaviour Finding 1 and Finding 5 recommend moving toward. Gating reads as well as writes behind authentication, and rejecting unauthenticated traffic outright, is the baseline a real SaaS API should meet before any of the more advanced controls (rate limiting, object-level authorization) are layered on top.

Remediation

- No remediation needed — referenced as the target behaviour for Findings 1 and 5.

Severity: **INFORMATIONAL**

6. Risk Classification Summary

The table below summarises severity across all findings.

#	Finding	Category	Severity
1	Unauthenticated PII exposure (/users)	Excessive Data Exposure / Broken Auth	**High**
2	Enumerable IDs, no object-level auth	Broken Object Level Authorization	**Medium**
3	Writes don't persist	Improper Inventory Management	**Low**
4	Loose input/type validation	Security Misconfiguration	**Low**
5	No rate limiting on reads	Unrestricted Resource Consumption	**Medium**
6	API key enforced on every call (ReqRes)	Reference / Positive Control	**Informational**

7. Authentication & Access Control Assessment

The two APIs reviewed sit at opposite ends of the authentication spectrum, which makes the comparison instructive on its own.

- JSONPlaceholder enforces no authentication on any endpoint, by design, as a learning/mock sandbox. Every record — including the full /users dataset analysed in Finding 1 — is reachable by anyone with the base URL.
- ReqRes enforces an API key on every single request, including simple reads, and fails closed with a clear error when no key is supplied.

For a real SaaS API, the ReqRes model is the correct baseline: authenticate every request by default, and treat "this is just a GET" as no excuse to skip access control. Object-level authorization (Finding 2) and rate limiting (Finding 5) should then be layered on top of that authenticated baseline, not used as a substitute for it.

8. Remediation Roadmap

Consolidated, in suggested priority order:

- Require authentication on every endpoint, including reads, before any other control is considered (addresses Finding 1, Finding 6 contrast).
- Add object-level authorization checks so an authenticated caller can only reach records they actually own or are permitted to view (Finding 2).
- Apply field-level response minimization so endpoints return only the fields a client needs, rather than full internal objects (Finding 1).
- Add rate limiting per IP/API key, with stricter limits on endpoints returning bulk or sensitive data (Finding 5).
- Enforce strict schema validation on all write endpoints and reject — rather than silently coerce — mismatched input (Finding 4).
- Confirm that responses reflect real persisted state, and document clearly which environments are live data stores versus mocks/sandboxes (Finding 3).

9. Conclusion

Testing two public demo APIs surfaced the same risk patterns that recur across real-world SaaS APIs: an endpoint that returns full personal data with no authentication at all, a request pattern that maps directly onto Broken Object Level Authorization, no throttling on bulk-data reads, and loose validation on writes. None of this required exploitation — every finding came from sending the kind of ordinary, read-only request a legitimate client would send, and observing what came back.

The contrast between the two APIs tested is itself the clearest takeaway: an API that requires a key for every call (ReqRes) closes off Finding 1 and Finding 5 by default, while one that requires nothing (JSONPlaceholder) is, by design, open to both. A real SaaS API should start from the authenticated-by-default end of that spectrum and add object-level authorization and rate limiting on top, rather than relying on obscurity or assuming GET requests are inherently low-risk.

Report prepared by Lakshya Mishra as part of the Future Interns Cyber Security Task 3 (2026).